DS n° 03 – Retouche d'images

- Faire tous les exercices dans un fichier NomPrenom.py à sauvegarder,
- mettre en commentaire l'exercice et la question traités (ex : # Exercice 1),
- ne pas oublier pas de commenter ce qui est fait dans votre code (ex : # Je crée une fonction pour calculer la racine d'un nombre),
- il est possible de demander un déblocage pour une question, mais celle-ci sera notée 0,
- il faut vérifier avant de partir que le code peut s'exécuter et qu'il affiche les résultats que vous attendez. Les lignes de code qui doivent s'exécuter sont décommentées.

Contexte

L'objectif de cette étude est de retoucher, à l'aide d'un script Python, une image en jouant sur la luminosité, le contraste et l'effet Bokeh (qui consiste à rendre flou l'arrière plan).

Le portrait que nous allons utiliser pour cette étude est celui de la Joconde de Léonard de Vinci, présent dans le dossier « Ressources » sous le nom « mona_lisa.png ».

1 Import d'une image et modifications de base

1.1 Charger et afficher une image

Question 1 **Copier** le code suivant pour charger l'image et y **ajouter** la commande `shape` afin d'afficher ses dimensions.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as img

img = img.imread("mona_lisa.png")
```

Question 2 **Afficher** l'image à l'aide des commandes `imshow(img)` et `show()` de la bibliothèque `matplotlib`.

1.2 Modifier une image

Nous allons dans la suite jouer sur la luminosité et le contraste de cette image.

Pour modifier ces paramètres, il faut appliquer la formule suivante à chaque valeur de couleur de chaque pixel : `valeur=contraste*valeur+luminosite`, avec `contraste` et `luminosite` des valeurs à modifier selon le réglage souhaité.

Attention, il faut veiller à ce que la valeur issue de ce calcul soit toujours comprise entre 0 et 1, sinon il y a aura une erreur à l'exécution. Pour cela, il faut utiliser la fonction `clip` de la bibliothèque `numpy`.

Question 3 **Créer** une fonction `contraste_luminosite(image,contraste,luminosite)` qui permet de modifier le contraste et la luminosité de l'image. **Copier** le code suivant pour la mettre en œuvre.

```
def contraste_luminosite(image,contraste,luminosite):
    ...

f, axarr = plt.subplots(1,2)
axarr[0].imshow(img)
axarr[0].set_title("Image originale")
axarr[0].axis('off')

axarr[1].imshow(contraste_luminosite(img,2,-0.5))
axarr[1].set_title("Nouvelle image")
axarr[1].axis('off')
plt.show()
```

2 Effet Bokeh numérique

2.1 Isoler l'arrière plan du sujet

L'objectif de cette partie est de mettre en valeur le sujet principal, Mona Lisa, en appliquant un flou à l'arrière-plan, tout en laissant le sujet net.

Pour cela, nous utiliserons un masque binaire. Ce masque a la forme d'une liste contenant, pour chaque pixel de l'image, la valeur :

— 1 si c'est un pixel du sujet (zone à laisser nette),

— 0 si c'est un pixel de l'arrière-plan (zone à flouter).

Ce masque est disponible dans le fichier « masque_mona_lisa.csv », le séparateur de colonne utilisé ici est « , ».

Question 4 **Importer** ce masque en créant une liste `mask` qui contient les valeurs du masque. **Afficher** la valeur de `mask[40]`.

2.2 Floutage de l'arrière plan

Dans la suite, il faut implémenter le flou. Cela revient à appliquer plusieurs fois un filtre moyenneur 3x3 sur chaque pixel. Ce flou moyenneur revient à remplacer la valeur d'un pixel par la moyenne des 9 pixels voisins (lui inclus) dans une fenêtre 3x3 centrée sur lui :

$$I'(i,j) = \frac{1}{9}\sum_{k=-1}^{1}\sum_{l=-1}^{1} I(i+k, j+l)$$

I(i-1,j-1)	I(i-1,j)	I(i-1,j+1)
I(i,j-1)	I(i,j)	I(i,j+1)
I(i+1,j-1)	I(i+1,j)	I(i+1,j+1)

Rappel $I(i,j) = [R_{(i,j)}, G_{(i,j)}, B_{(i,j)}]$ est un vecteur dont les composantes représentent les valeurs des couleurs Rouge, Vert, Bleu du pixel.

Question 5 **Coder** la fonction `moyenne_voisins(img,i,j)` qui permet de calculer cette moyenne.
`moyenne_voisins(img,75,11)` doit renvoyer `[0.3381264  0.28496733 0.18649238]`.

En appliquant cette opération plusieurs fois :

— les détails fins de l'arrière-plan disparaissent progressivement,

— l'effet de flou devient plus prononcé,

— le sujet principal reste net grâce au masque.

Le code suivant permet d'appliquer le flou sur l'arrière plan de la Joconde.

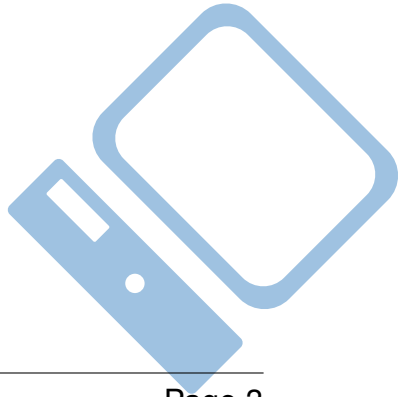
```
def flou(img, n):
    img_blur = img.copy()
    h, w = img.shape[:2]
    for iter in range(n):
        for i in range(1,h-1):
            for j in range(1,w-1):
                if mask[i,j]==.....:
                    img_blur[i,j] = moyenne_voisins(img,i,j)
    return img_blur

fig, axes = plt.subplots(1, 2)
axes[0].imshow(img)
axes[0].set_title("Image originale")
axes[0].axis('off')
axes[1].imshow(flou(img, n=2))
axes[1].set_title("Arrière-plan flou")
axes[1].axis('off')
plt.show()
```

Question 6 **Remplacer** les `.....` dans le code afin que l'effet Bokeh soit appliqué sur l'arrière plan de l'image.

Question 7 **Créer** une fonction `flou_recursif(img, n)` qui effectue le mettre travail que la fonction `flou(img, n)` mais en utilisant un algorithme récursif. **Afficher** l'image obtenue par cette nouvelle fonction.

FIN



Correction

Question 1

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as img

img = img.imread("mona_lisa_1.png")
print(img.shape)
```

Question 2

```
plt.imshow(img)
plt.axis('off')
plt.show()
```

Question 3

```
def contraste_luminosite(image,contraste,luminosite):
    return np.clip(contraste*image+luminosite,0,1)

f, axarr = plt.subplots(1,2)
axarr[0].imshow(img)
axarr[0].set_title("Image originale")
axarr[0].axis('off')

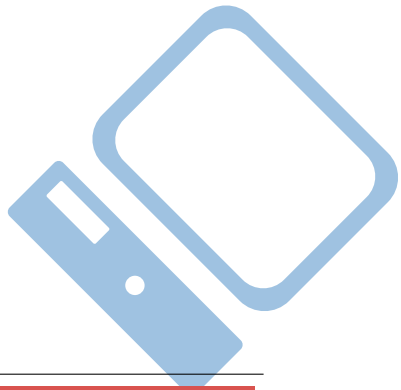
axarr[1].imshow(contraste_luminosite(img,2,-0.5))
axarr[1].set_title("Nouvelle image")
axarr[1].axis('off')
plt.show()
```

Question 4

```
fichier = open("masque_mona_lisa.csv", 'r')
contenu = fichier.read()
fichier.close()
lignes = contenu.split("\n")
mask=[]

for ligne in lignes[:-1]:
    data=[int(i) for i in ligne.split(',')]
    mask.append(data)
mask=np.array(mask)

print(mask[40])
```



Question 5

```
def moyenne_voisins(img,i,j):
    pixel = 0
    for k in range(-1,2):
        for l in range(-1,2):
            pixel += (1/9)*img[i+k,j+l]
    return pixel

print(moyenne_voisins(img,75,11))
```

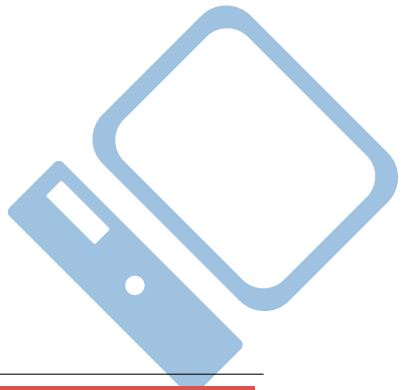
Question 6

```
def flou(img, n):
    img_blur = img.copy()
    h, w = img.shape[:2]
    for iter in range(n):
        for i in range(1,h-1):
            for j in range(1,w-1):
                if mask[i,j]==0:
                    img_blur[i,j] = moyenne_voisins(img,i,j)
    return img_blur

fig, axes = plt.subplots(1, 2)
axes[0].imshow(img)
axes[0].set_title("Image originale")
axes[0].axis('off')
axes[1].imshow(flou(img, n=2))
axes[1].set_title("Arrière-plan flou")
axes[1].axis('off')
plt.show()
```

Question 7

```
def flou_recursif(img, n):
    if n == 0:
        return img
    else:
        h, w = img.shape[:2]
        img_blur = img.copy()
        for i in range(1,h-1):
            for j in range(1,w-1):
                if mask[i,j]==0:
                    img_blur[i,j] = moyenne_voisins(img,i,j)
        return flou_recursif(img_blur, n-1)
```



Correction

```
fig, axes = plt.subplots(1, 2)
axes[0].imshow(img)
axes[0].set_title("Image originale")
axes[0].axis('off')
axes[1].imshow(flou_recursif(img, n=2))
axes[1].set_title("Arrière-plan flou")
axes[1].axis('off')
plt.show()
```

